대회 / 코딩테스트 알고리즘 정리

Seojin Kim[1]

March 11, 2026

[1]Department of Computer Engineering, Sejong University, Seoul, Republic of Korea. GitHub: github.com/rlatjwls7882

머리말

이 문서는 **GitHub 저장소**에서 제작·배포되며, 최신 버전은 항상 해당 저장소에서 확인할 수 있습니다.

본 문서는 **알고리즘의 동작 방식과 구현**을 중심으로 설명하여, 학습 및 복습 시 빠르게 참고할 수 있는 자료를 목표로 합니다.

차례

머리말 **i**

1 기본 알고리즘 **1**

1.1 Binary Search (이분 탐색) 1

1.2 Prefix Sum (누적 합) 4

1.3 Two Pointer (두 포인터) 5

 CC BY-NC-SA 4.0

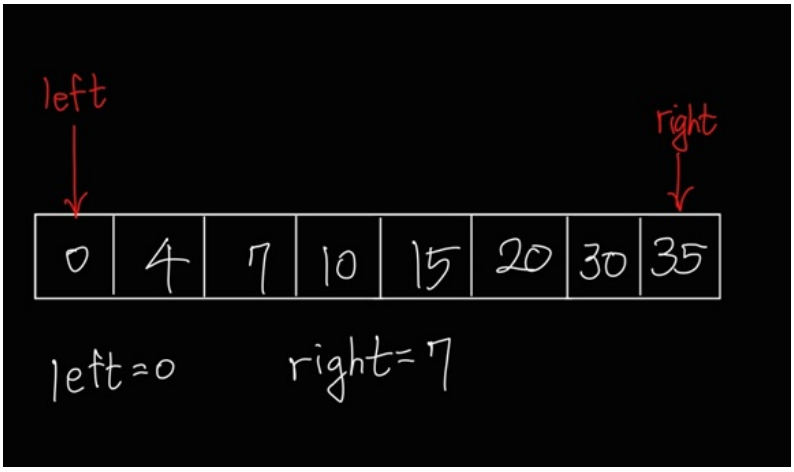
1장

기본 알고리즘

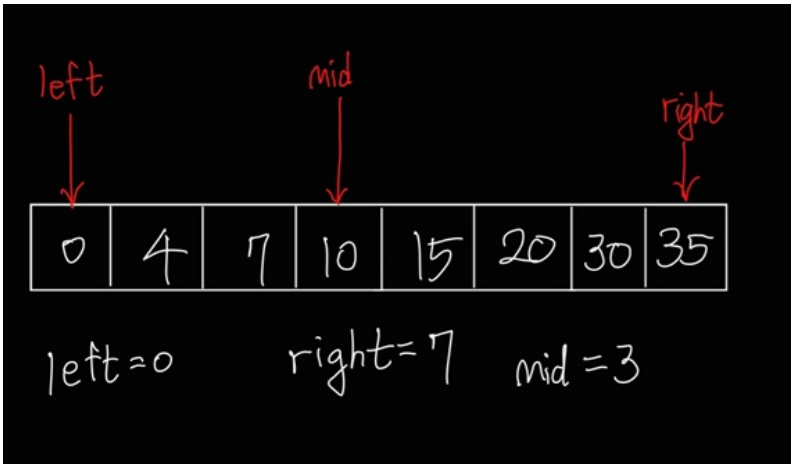
1.1 Binary Search (이분 탐색) Silver IV

정렬된 데이터에서 탐색 범위를 절반씩 줄여가며 원하는 값을 찾는 알고리즘

시간복잡도: $O(M \log N)$ (N: 데이터 개수, M: 탐색 횟수)

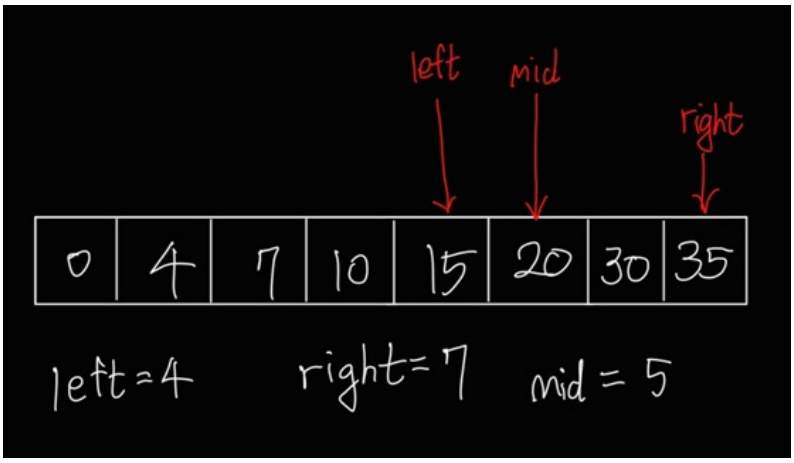


정렬된 길이 7의 배열 A에서 값 21을 찾는다고 가정하자.
초기에 21이 존재할 수 있는 인덱스 범위는 $0 \leq i \leq 6$이다.

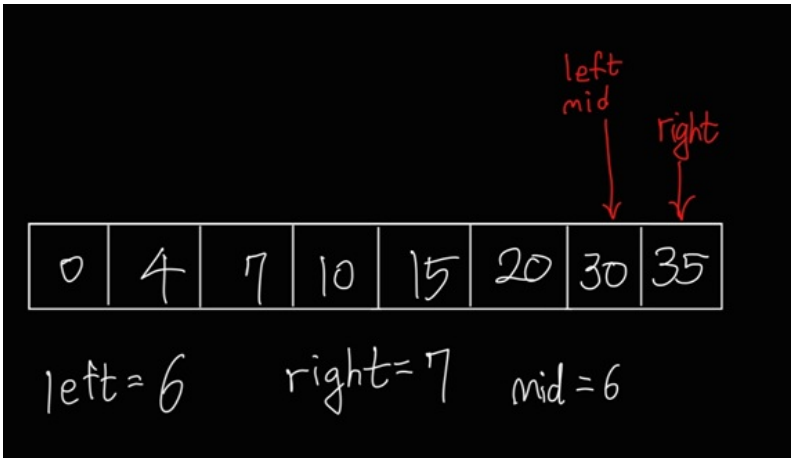


 CC BY-NC-SA 4.0

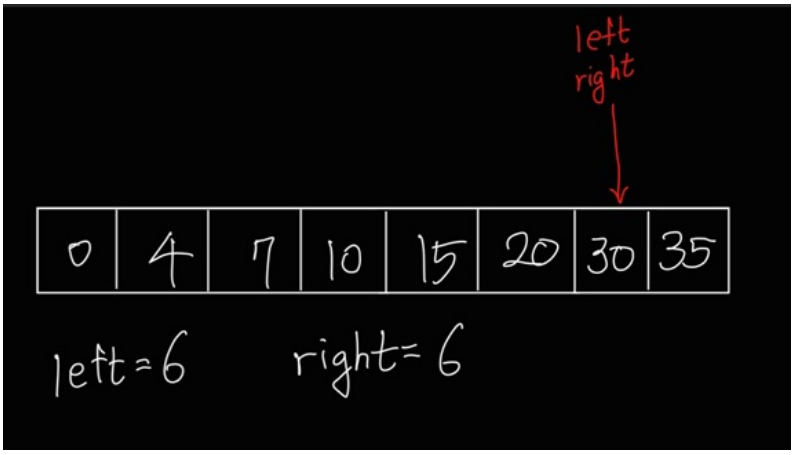
현재 탐색 범위의 가운데 값을 확인한다. $mid = \lfloor \frac{left+right}{2} \rfloor = 3$
이때 $A_{mid} < 21$이므로 $left \le i \le mid$인 모든 A_i가 21보다 작다는 것을 알 수 있다.
따라서 탐색 범위를 $left = mid + 1$로 갱신한다.



다시 탐색 범위의 가운데 값을 확인한다. $mid = \lfloor \frac{left+right}{2} \rfloor = 5$
여번에도 $A_{mid} < 21$이므로 $left \le i \le mid$인 모든 A_i가 21보다 작다는 것을 알 수 있다.
따라서 탐색 범위를 $left = mid + 1$로 갱신한다.



다시 탐색 범위의 가운데 값을 확인한다. $mid = \lfloor \frac{left+right}{2} \rfloor = 6$
이번에는 $A_{mid} \ge 21$이므로 $mid \le i \le right$인 모든 A_i가 21보다 크거나 같다는 것을 알 수 있다.
따라서 탐색 범위를 $right = mid$로 갱신한다.



이제 $left = right$이기 때문에 더이상 범위를 줄일 수 없다.
마지막으로 $A_{left} = A_{right}$ 값을 확인한다.
이 값이 21이 아니기 때문에 배열 A에는 21이 존재하지 않는다.

연습 문제: 백준 1920번

```
/** http://boj.kr/13586b0c1d744fe8b0c4f6a8393ed65c 제출 코드 */
#include<bits/stdc++.h>
using namespace std;

int A[100'001];

/**
 * Binary Search
 * 처음 범위 : 0 ~ n-1
 *
 * right가 left보다 큰 동안 :
 * - mid를 (left+right)/2로 설정
 * - mid 위치의 값이 현재 탐색중인 값보다 작다면 left = mid+1
 * - 그렇지 않다면 right = mid
 */
bool binary_search(int val) {
    int left=0, right=n-1;
    while(left<right) {
        int mid = (left+right)/2;
        if(A[mid]<val) left=mid+1;
        else right=mid;
    }
    return A[left] == val;
}

int main() {
    cin.tie(0)->sync_with_stdio(0);
    int n; cin >> n;
    for(int i=0;i<n;i++) cin >> A[i];
    sort(A, A+n);

    int m; cin >> m;
    while(m--) {
        int val; cin >> val;
```

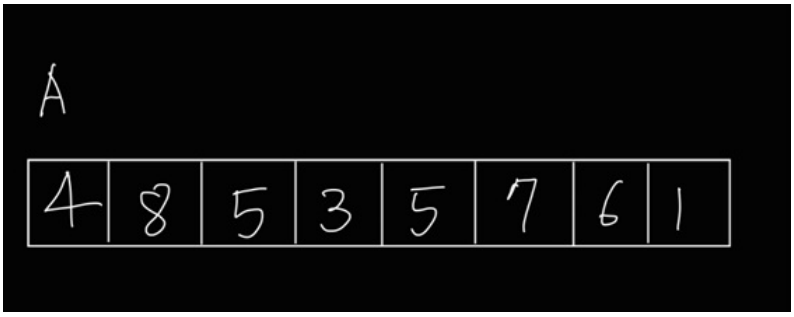
 CC BY-NC-SA 4.0

```
        cout << binary_search(val) << '\n';
    }
}
```

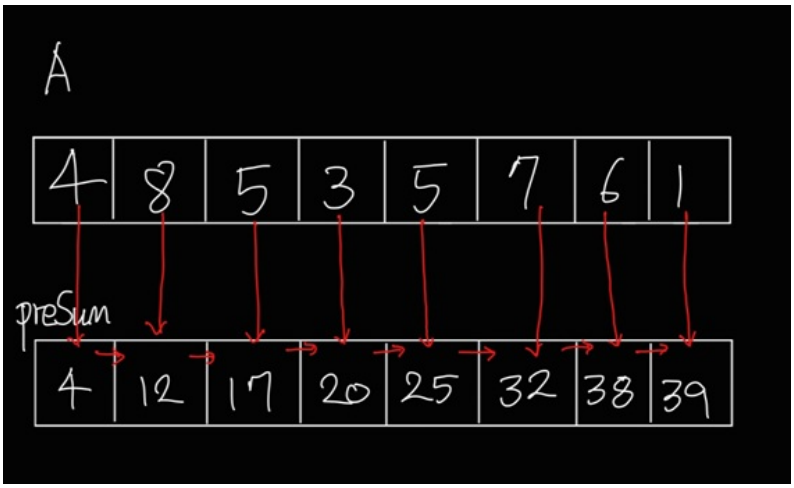
1.2 Prefix Sum (누적 합) Silver III

배열의 누적 합을 미리 계산하여 임의 구간의 합을 $O(1)$에 구하는 알고리즘

시간복잡도: 전처리 $O(N)$, 쿼리 $O(1)$ (N: 데이터 개수)



배열 A가 다음과 같다고 하자.



이때 배열 A의 누적 합 배열 $preSum$을 다음과 같이 정의한다.

$$preSum_i = A_0 + A_1 + \cdots + A_i$$

즉, $preSum_i$는 배열의 첫 번째 원소부터 i번째 원소까지의 합을 의미한다.
이때 $A_i + A_{i+1} + \cdots + A_j$는 다음과 같이 계산할 수 있다.

$$(A_0 + A_1 + \cdots + A_j) - (A_0 + A_1 + \cdots + A_{i-1})$$

이는 누적 합 배열을 이용하면 $preSum_j - preSum_{i-1}$로 표현할 수 있다.

연습 문제: 백준 11659번

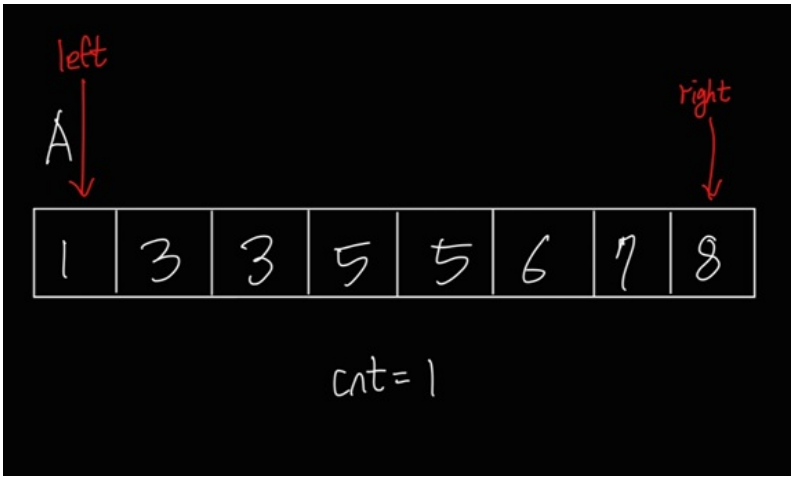
```
/** http://boj.kr/8a8bf787cca8473fa1b0c13219b94918 제출 코드 */
#include<bits/stdc++.h>
using namespace std;

// MAX : 수의 최대 개수
const int MAX = 100'001;

// preSum[i] : 1번 수부터 i번 수까지의 누적 합
int preSum[MAX];

int main() {
    cin.tie(0)->sync_with_stdio(0);
    int n, m; cin >> n >> m;
    for(int i=1;i<=n;i++) {
        cin >> preSum[i]; // i번째 수 입력
        preSum[i] += preSum[i-1]; // i번째 수에 0 ~ i-1 수의 합 더하기 -> 0 ~ i 수의 합
        ↪ 완성
    }

    while(m--) {
        int i, j; cin >> i >> j;
        cout << preSum[j] - preSum[i-1] << '\n'; // i ~ j 수의 합 = (0 ~ j 수의 합) -
        ↪ (0 ~ i-1 수의 합)
    }
}
```

1.3 Two Pointer (두 포인터) Silver III

두 개의 포인터를 움직이며 배열에서 조건을 만족하는 쌍을 효율적으로 찾는 알고리즘

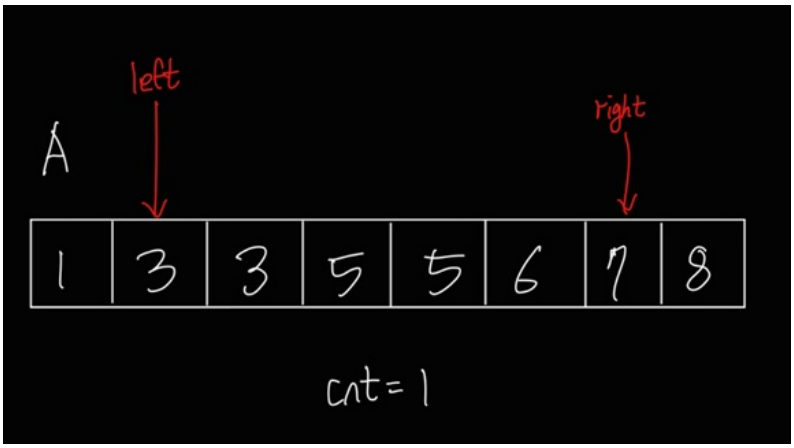
시간복잡도: $O(N)$ (N: 데이터 개수)

다음과 같은 배열 A에서 두 수의 합이 9가 되는 쌍을 찾는다고 하자.

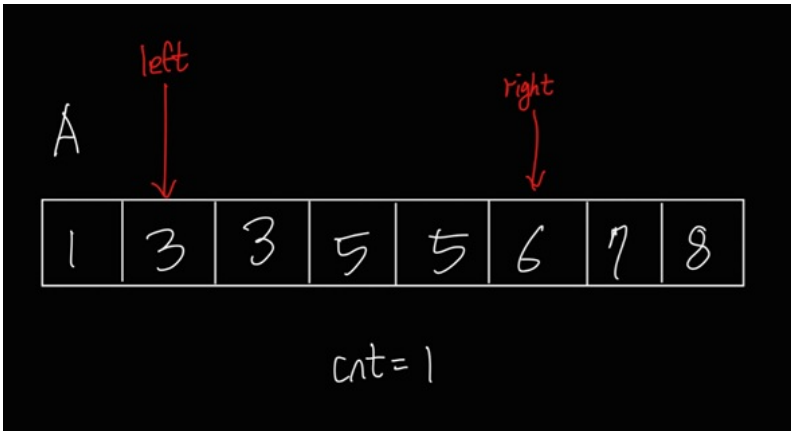


처음에는 왼쪽 포인터 $left$를 배열의 왼쪽 끝에, 오른쪽 포인터 $right$를 배열의 오른쪽 끝에 두고 탐색을 시작한다.

$A_{\text{left}} + A_{\text{right}} = 9$이므로 cnt를 1 증가시키고, 사용한 조합을 제외하기 위해 $left$를 1 증가시키며 $right$를 1 감소시킨다.

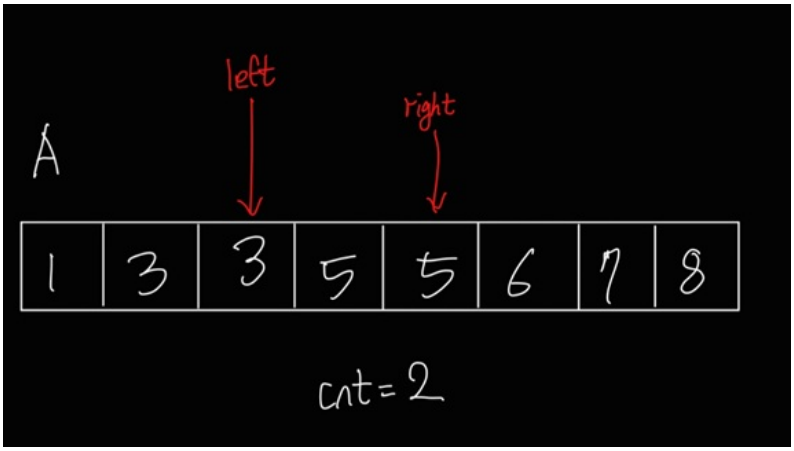


$A_{\text{left}} + A_{\text{right}} > 9$이므로 합을 줄이기 위해 $right$를 1 감소시킨다.

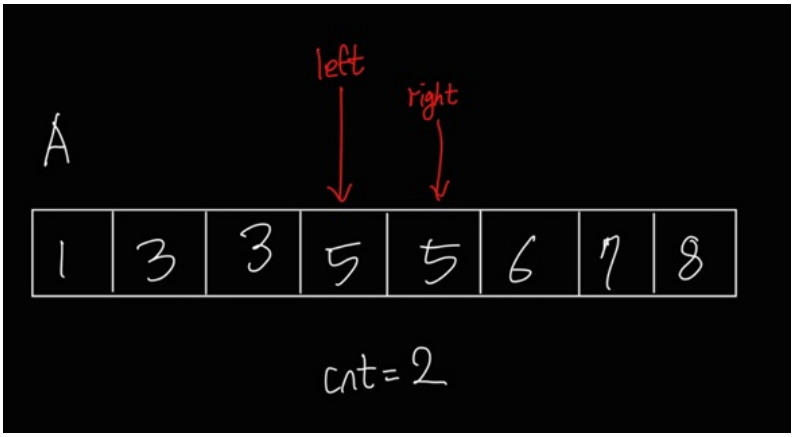


$A_{\text{left}} + A_{\text{right}} = 9$이므로 cnt를 1 증가시키고, 사용한 조합을 제외하기 위해 $left$를 1 증가시키며 $right$를 1 감소시킨다.

 CC BY-NC-SA 4.0



$A_{\text{left}} + A_{\text{right}} < 9$이므로 합을 늘리기 위해 $left$를 1 증가시킨다.



$A_{\text{left}} + A_{\text{right}} > 9$이므로 $right$를 1 감소시킨다. $left \geq right$가 되었기에 탐색을 종료한다.
이렇게 $left < right$인 동안 위 과정을 반복하면, 두 수의 합이 9가 되는 모든 쌍을 찾을 수 있다.

연습 문제: 백준 3273번

```
/** http://boj.kr/bba07bce85944877bf61c2508cd6a062 제출 코드 */
#include<bits/stdc++.h>
using namespace std;

int a[100'000];

int main() {
    cin.tie(0)->sync_with_stdio(0);
    int n; cin >> n;
    for(int i=0;i<n;i++) cin >> a[i];
    int val; cin >> val;
    sort(a, a+n); // 정렬

    int cnt=0, left=0, right=n-1; // left와 right는 각각 왼쪽 끝에 위치한 포인터, 오른쪽
    ↪ 끝에 위치한 포인터
    while(left<right) {
        if(a[left]+a[right]<val) { // 두 원소의 합이 val보다 작으면
```

```
            left++; // 왼쪽 포인터를 오른쪽으로 이동
        } else if(a[left]+a[right]>val) { // 두 원소의 합이 val보다 크면
            right--; // 오른쪽 포인터를 왼쪽으로 이동
        } else { // 두 원소의 합이 val과 동일하면
            cnt++;
            left++; // 왼쪽 포인터를 오른쪽으로, 오른쪽 포인터를 왼쪽으로 이동 (사용한 원소
            ↪ 제외)
            right--;
        }
    }
    cout << cnt;
}
```

 CC BY-NC-SA 4.0